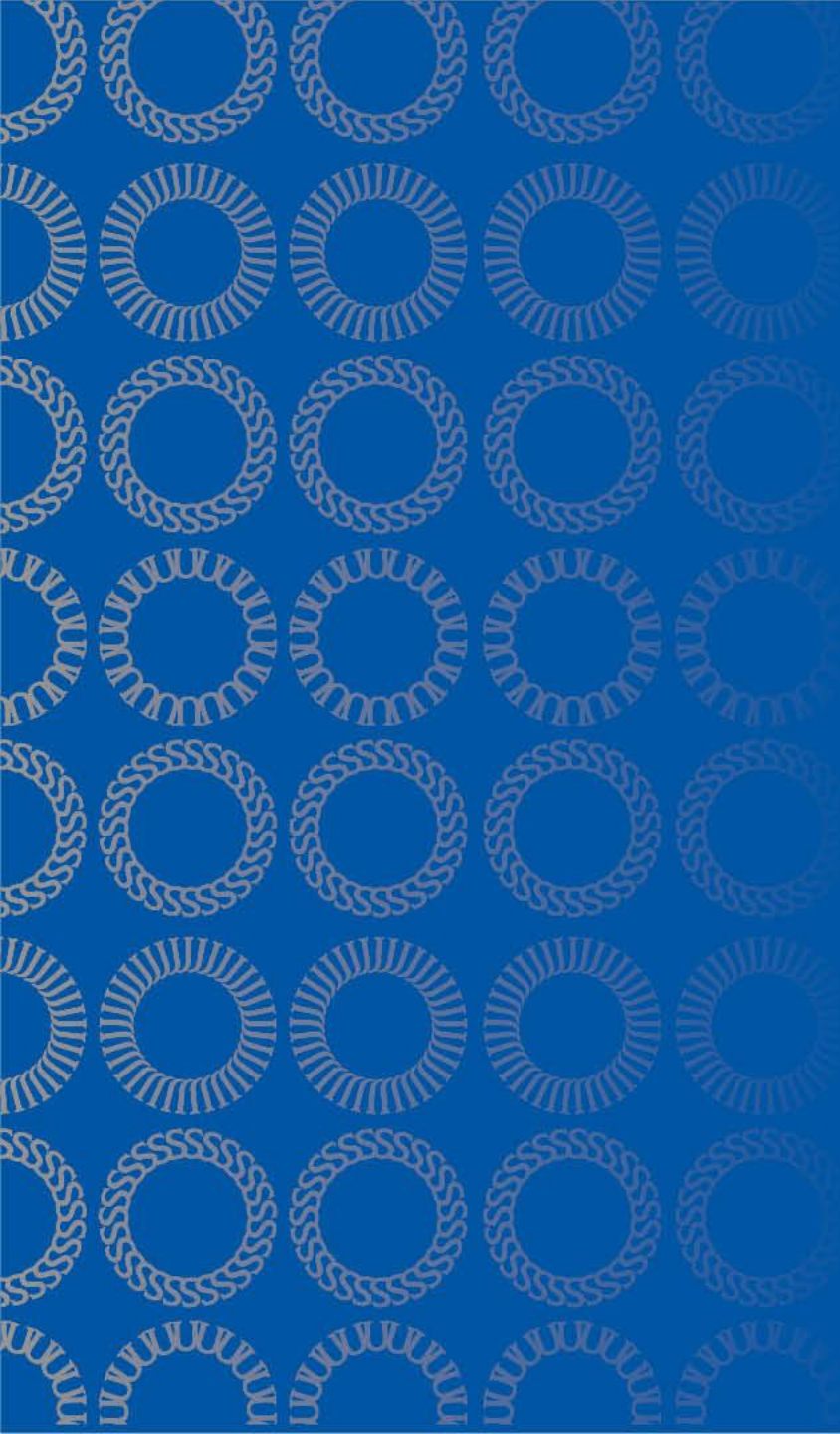




DATA 255 Deep Learning Technologies

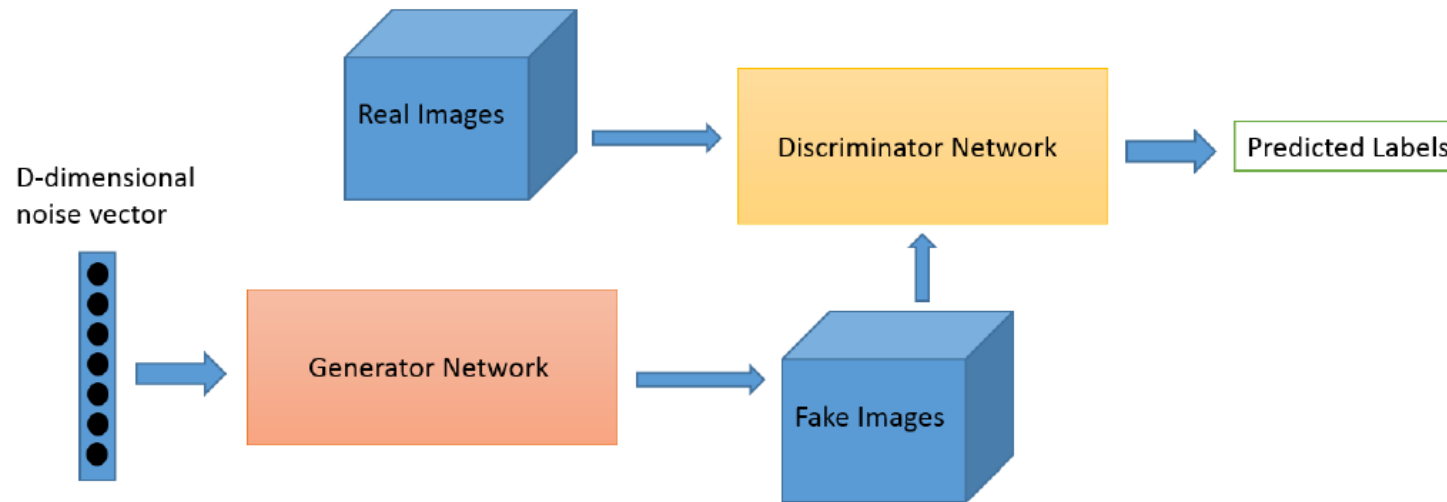
Nov 6, 2025

Taehee Jeong, Ph.D.

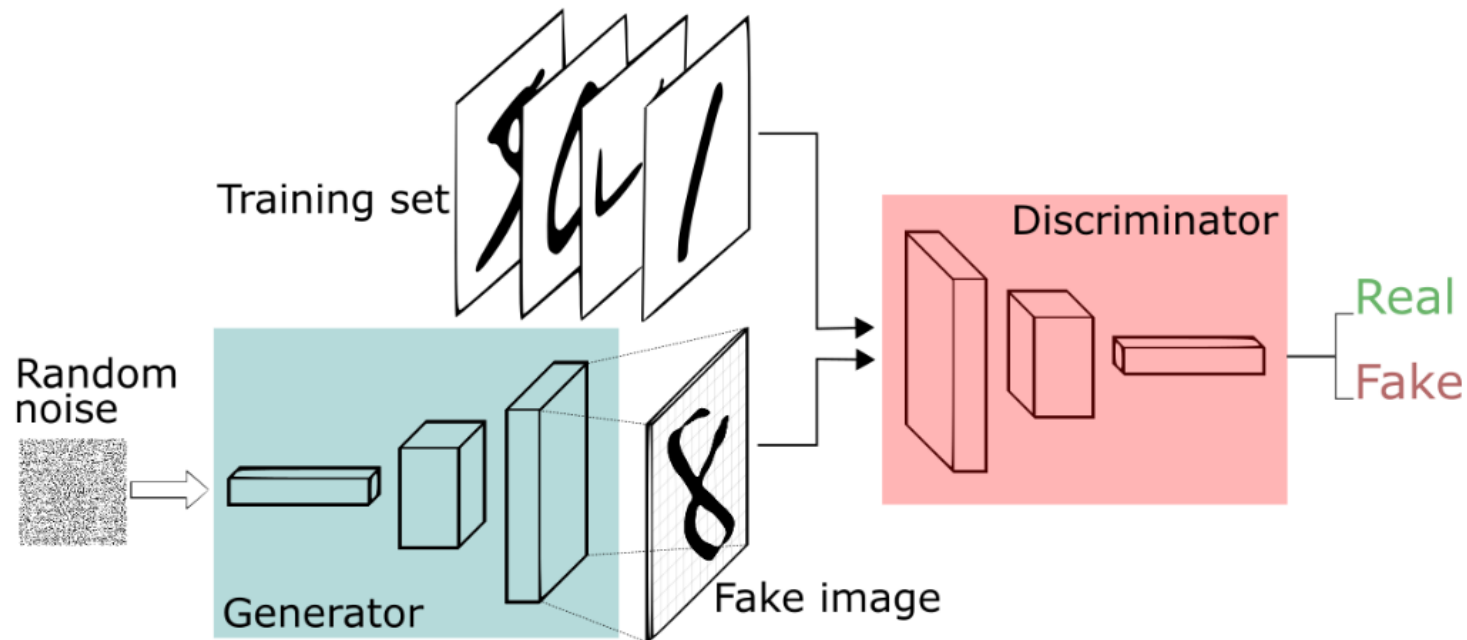
Previous topics

- Generative Adversarial Networks (GAN)

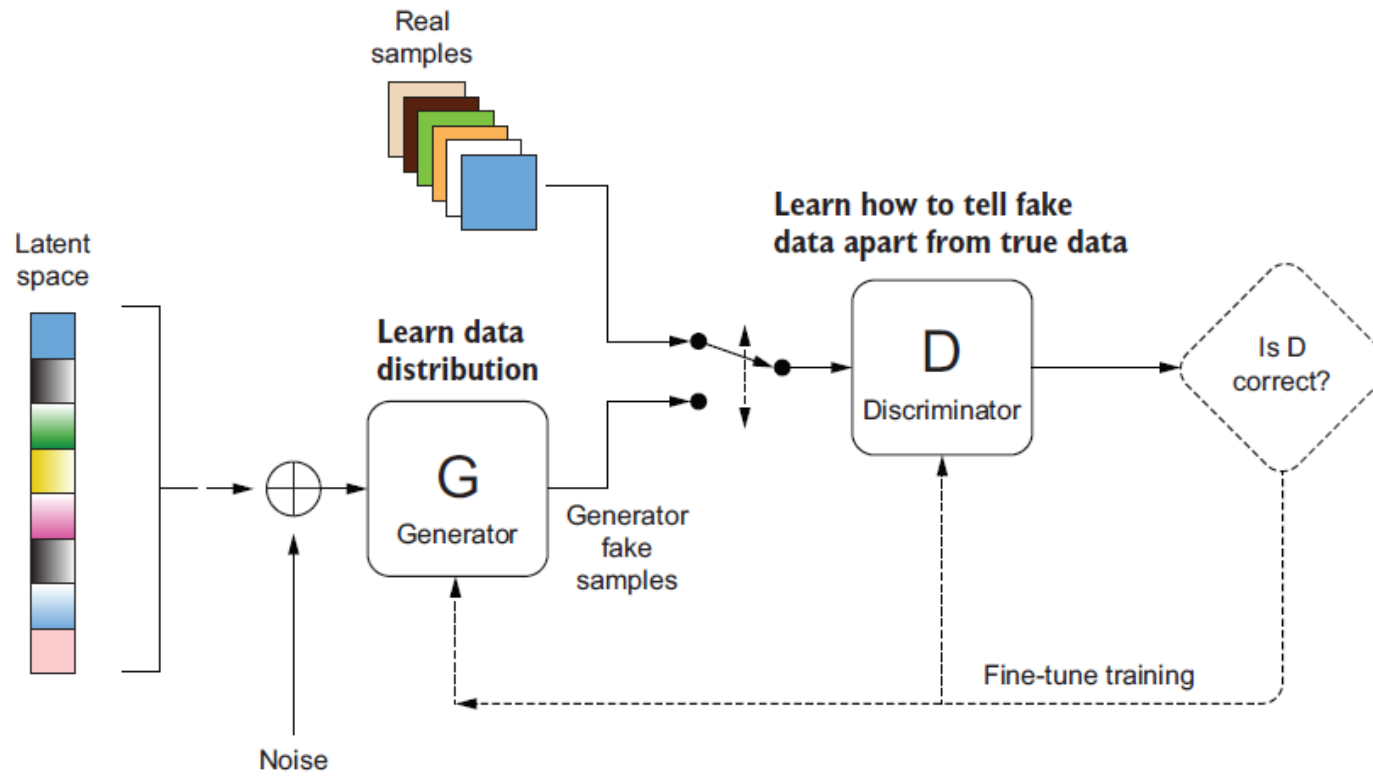
Generative Adversarial Networks (GAN)



Generative Adversarial Networks (GAN)

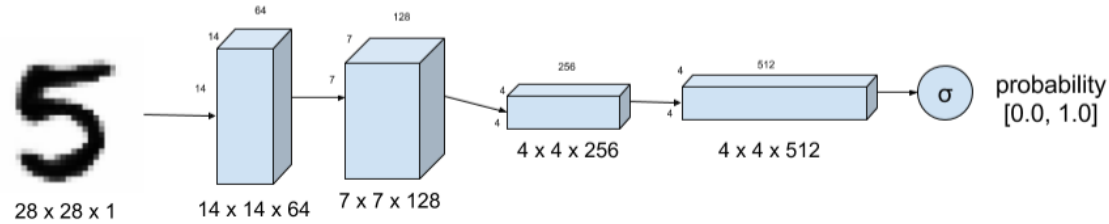


Generative Adversarial Networks (GAN)

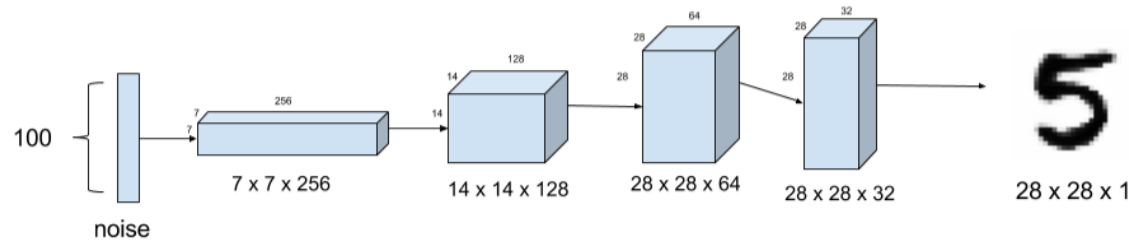


Vallina GAN

Discriminator



Generator

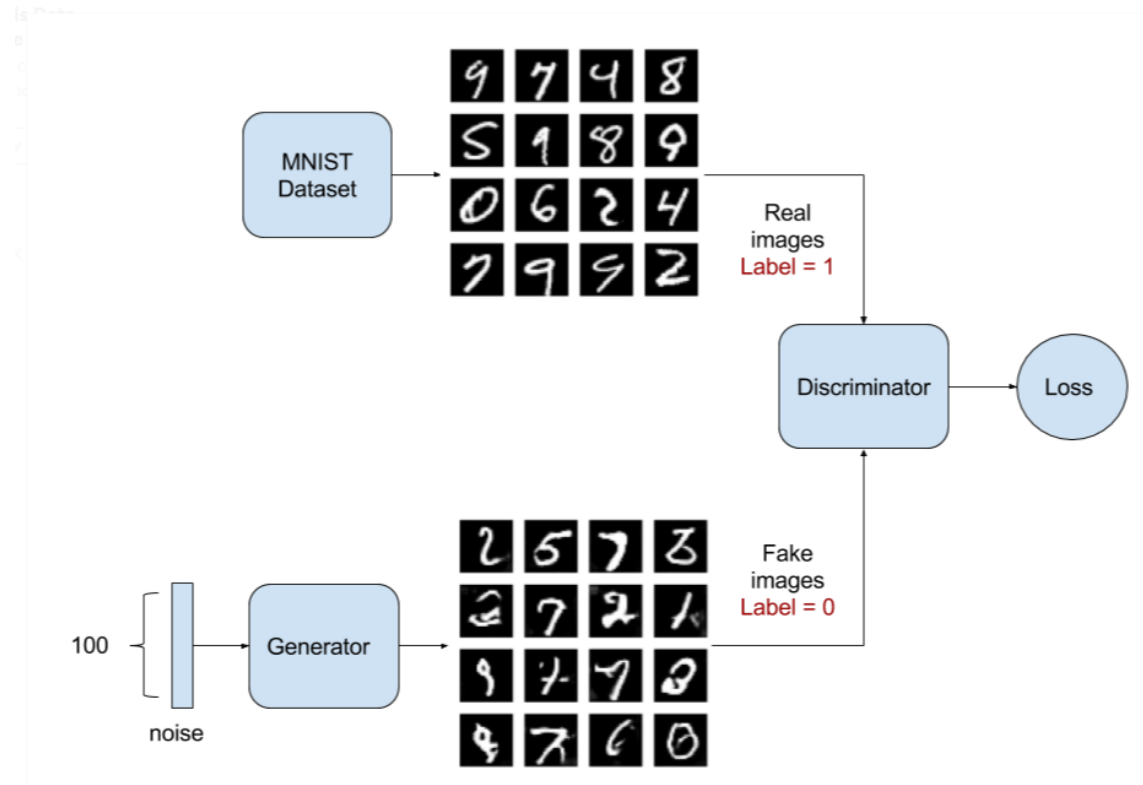


Adversarial Model



Vallina GAN

Training



Generator and Discriminator in GAN

	Generator	Discriminator
Input	A vector of random numbers	The Discriminator receives input from two sources: ■ Real examples coming from the training dataset ■ Fake examples coming from the Generator
Output	Fake examples that strive to be as convincing as possible	Predicted probability that the input example is real
Goal	Generate fake data that is indistinguishable from members of the training dataset	Distinguish between the fake examples coming from the Generator and the real examples coming from the training dataset

Conflicting objectives

- The Discriminator's goal is to be as accurate as possible.
- The Generator seeks to fool the Discriminator by producing fake examples that are indistinguishable from the real data.

Train GAN

```
##Train discriminator with real images and labels
discriminator_optimizer.zero_grad()
real_labels = torch.ones(real_images.size(0), 1, device=device)
real_outputs = discriminator(real_images)
real_loss = criterion(real_outputs, real_labels)
real_loss.backward()

##Train discriminator with fake images generated by the generator.
#Generates random noise vectors to feed the generator.
noise = torch.randn(real_images.size(0), NOISE_DIM, device=device)
fake_images = generator(noise)
fake_labels = torch.zeros(real_images.size(0), 1, device=device)
fake_outputs = discriminator(fake_images.detach())
fake_loss = criterion(fake_outputs, fake_labels)
fake_loss.backward()
discriminator_optimizer.step()
dis_totalloss += real_loss.item() * real_images.size(0) + fake_loss.item() * real_images.size(0)

##Creates labels of ones to trick the discriminator into thinking generated images are real.
generator_optimizer.zero_grad()
fake_labels = torch.ones(real_images.size(0), 1, device=device)
fake_outputs = discriminator(fake_images)
gen_loss = criterion(fake_outputs, fake_labels)
gen_loss.backward()
generator_optimizer.step()
gen_totalloss += gen_loss.item() * real_images.size(0)
```

Step 1: Train Discriminator with real images and labels

Step 2: Train Discriminator with fake images generated by Generator and labels

Step 3: Trick discriminator with fake images and fake labels → Train Generator

Today's Topics

- Diffusion model:
 - Forward diffusion process
 - Noising process
 - Reverse diffusion process

Why Diffusion model?

It generates higher-quality images than those produced by GANs.

DALL·E 3: text-to-image synthesis



[illegible]

SU SAN JOSÉ STATE
UNIVERSITY

Stable diffusion 3



Idea of diffusion model

*“The essential idea, inspired by non-equilibrium statistical physics, is to systematically and **slowly destroy structure in a data distribution** through an **iterative forward diffusion process**. We then learn a **reverse diffusion process** that **restores structure in data**, yielding a highly flexible and tractable generative model of the data.”*

- The diffusion process is split into forward and reverse diffusion processes.
- The forward diffusion process is a process of turning an image into noise.
- The reverse diffusion process is supposed to turn that noise into the image again.

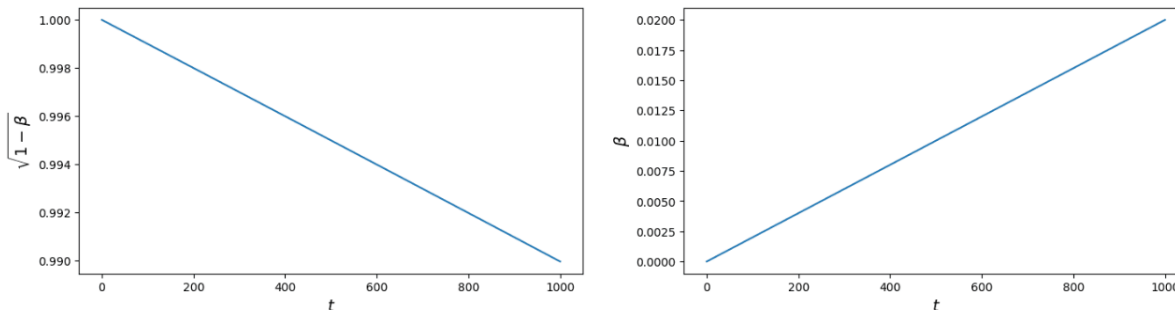
Forward process or diffusion process

Gradually adds Gaussian noise to the data according to a variance schedule $\beta_1; \dots; \beta_T$:

$$q(x_t|x_{t-1}) = N(x_t, \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

q is forward process, x_t is the output of the forward process at step t , x_{t-1} is the input at step t .

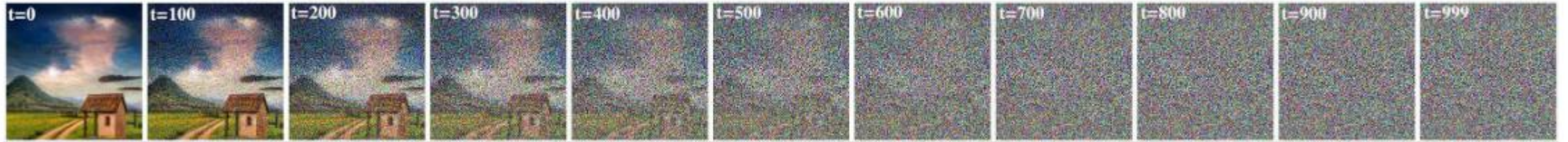
N is a normal distribution, $\sqrt{1 - \beta_t}x_{t-1}$ is mean, $\beta_t I$ is a variance.



In the case $\beta_t: 0.0001 \sim 0.02$

Linear schedule

β_t is called schedule and values can range from 0 to 1.



$$q(x_t|x_{t-1}) = N(x_t, \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

- linear schedule is not efficient.

Cosine schedule



- Cosine schedule allowed to reduce the number of steps to 50.

Noising process

Adding noise to the image using a forward diffusion process is going to be slow.

The training process doesn't use examples in line with the forward process but rather it uses samples from arbitrary timestep t .

At each training step we would need to iterate through t steps to generate 1 training sample.

- the entire noise to be added at T

$$q(x_{1:T}|x_0) := \prod_{t=1}^T q(x_t|x_{t-1})$$

$$\prod_{t=1}^T q(x_t|x_{t-1}) \approx q_t(q_{t-1}(q_{t-2}(\cdots q_1(x_0))))$$

noising process

- $q(x_1|x_0) = N(x_1, \sqrt{1 - \beta_1}x_0, \beta_1 I)$
 - $q(x_2|x_1) = N(x_2, \sqrt{1 - \beta_2}x_1, \beta_2 I)$
-
- $q(x_1|x_0) = N(x_1, \sqrt{\alpha_1}x_0, (1 - \alpha_1)I)$
 - $q(x_t|x_0) \approx N(x_t, \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

$$\sqrt{\alpha_t}x_{t-1} = \sqrt{\alpha_t} * \sqrt{\alpha_{t-1}}x_{t-2} = \sqrt{\alpha_t \alpha_{t-1}}x_{t-2}$$

$$\sqrt{\alpha_t}x_{t-1} = \sqrt{\alpha_t \alpha_{t-1} \cdots \alpha_1}x_0 = \sqrt{\bar{\alpha}_t}x_0$$

noising process

- $q(x_t|x_0) \approx N(x_t, \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$

- $x_t \approx \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$

$$\epsilon \sim N(0,1)$$

- we can calculate noise at any arbitrary step t without going through the process.

Reparameterization trick:

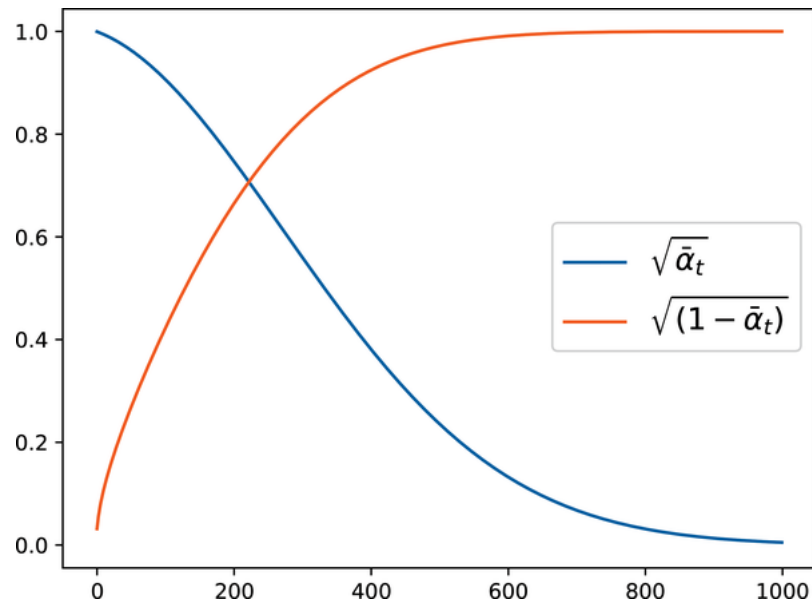
$$N(\mu, \sigma^2) = \mu + \sigma \cdot \epsilon$$

$$\bar{\alpha}_t = \alpha_t \alpha_{t-1} \cdots \alpha_1$$

$$\alpha_t = 1 - \beta_t$$

noising process

- $$x_t \approx \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$
$$\epsilon \sim N(0,1)$$



In the case $\beta_t: 0.001 \sim 0.02$

Reverse diffusion process

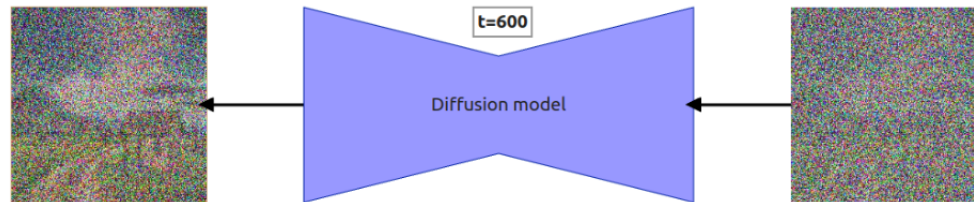
the goal is to convert pure noise into an image.

Incorrect:

- diffusion model removes noise from an input image.
- It predicts the noise to be removed from an input.

Correct:

- It predicts the entire noise to be removed with a given timestep



Reverse diffusion process

$$\alpha_t = 1 - \beta_t$$

$$\sigma_t^2 = \beta_t$$

- $x_{t-1} \approx N(x_{t-1}, \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(x_t, t)), \sqrt{\beta_t} \epsilon)$

- $x_{t-1} \approx \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$

$\epsilon_\theta(x_t, t)$: model's output (predicted noise)

$z \sim N(0,1)$

Adding noise

Gaussian noise is a type of noise that follows a normal distribution.

1. Generate Gaussian noise of the same dimensions as the input images

```
x = next(iter(train_dataloader))["images"][:8]
noise = torch.rand_like(x)
```

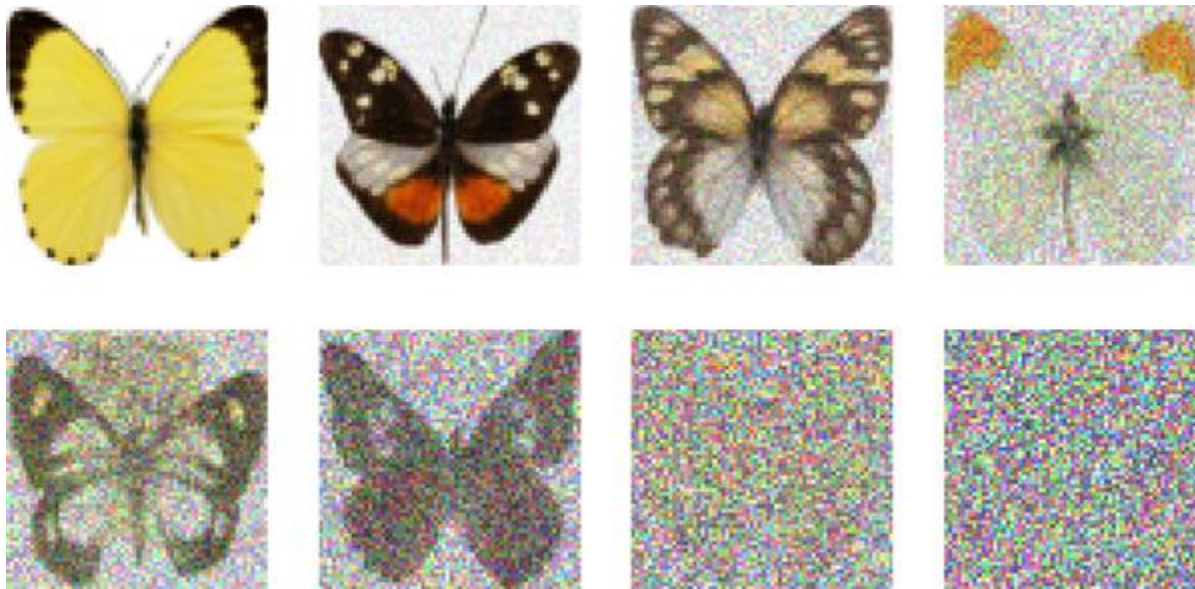
2. Add varying amounts of noise to the input images

```
def corrupt(x, noise, amount):
    # Reshape amount so it works correctly with the original data
    amount = amount.view(-1, 1, 1, 1) # make sure it's broadcastable

    # Blend the original data and noise based on the amount
    return (
        x * (1 - amount) + noise * amount
    ) # equivalent to x.lerp(noise, amount)
```

Adding noise

```
amount = torch.linspace(0, 1, 8)  
noised_x = corrupt(x, noise, amount)  
show_images(noised_x * 0.5 + 0.5)
```



Noise Schedule

The amount of noise to add is controlled by a noise schedule.

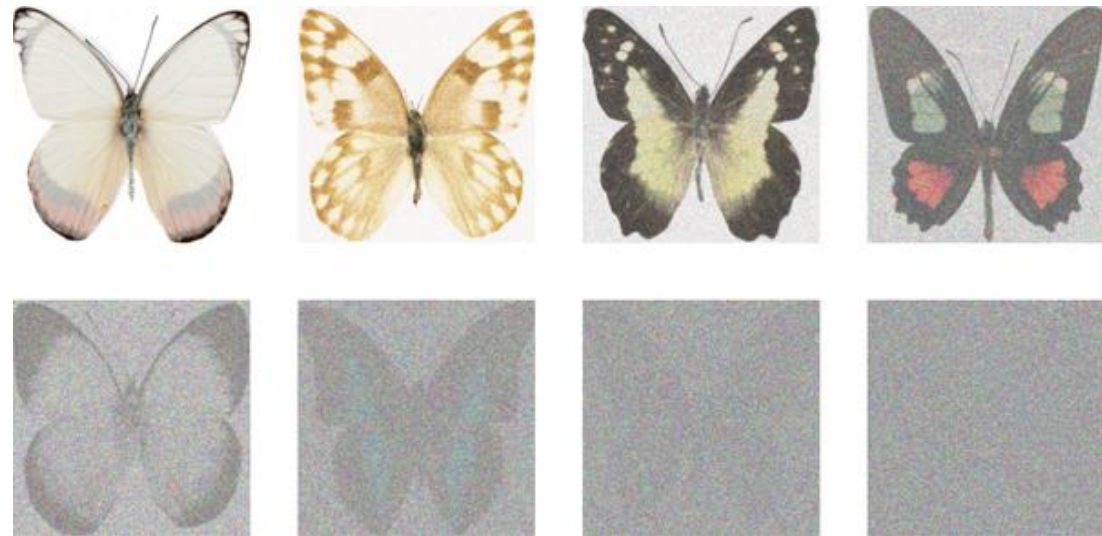
Noise scheduler takes in a batch of images and a list of timesteps and determines how to create the noisy versions of those images.

```
from diffusers import DDPMScheduler

# We'll learn about beta_start and beta_end in the next sections
scheduler = DDPMScheduler(
    num_train_timesteps=1000, beta_start=0.001, beta_end=0.02
)

# Create a tensor with 8 evenly spaced values from 0 to 999
timesteps = torch.linspace(0, 999, 8).long()

# We load 8 images from the dataset and
# add increasing amounts of noise to them
x = batch["images"][:8]
noise = torch.rand_like(x)
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```



DDPM Scheduler

```
class DDPM(nn.Module):
    def __init__(self, network, n_steps=1000, min_beta=10 ** -4, max_beta=0.02, device=None, image_chw=(1, 28, 28)):
        super(DDPM, self).__init__()
        self.n_steps = n_steps
        self.device = device
        self.image_chw = image_chw
        self.network = network.to(device)
        self.betas = torch.linspace(min_beta, max_beta, n_steps).to(device)
        self.alphas = 1 - self.betas
        self.alpha_bars = torch.tensor([torch.prod(self.alphas[:i + 1]) for i in range(len(self.alphas))]).to(device)

    def forward(self, x0, t, eta=None):
        # Make input image more noisy (we can directly skip to the desired step)
        n, c, h, w = x0.shape
        a_bar = self.alpha_bars[t]

        if eta is None:
            eta = torch.randn(n, c, h, w).to(self.device)

        noisy = a_bar.sqrt().reshape(n, 1, 1, 1) * x0 + (1 - a_bar).sqrt().reshape(n, 1, 1, 1) * eta
        return noisy

    def backward(self, x, t):
        # Run each image through the network for each timestep t in the vector t.
        # The network returns its estimation of the noise that was added.
        return self.network(x, t)
```

Train model

1. Load a batch of images,
2. Add noise to the images,
3. Feed the noisy images into the model,
4. Predict noise
5. Calculate the loss.
6. Backpropagate and update weights with the optimizer.

```
for epoch in range(num_epochs):
    model.train()
    train_loss, total = 0, 0
    for batch_x, _ in train_loader:

        # Load the input images
        clean_images = batch_x.to(device)    #(64,1,28,28)
        n = len(clean_images)

        # Sample noise to add to the images
        eta = torch.randn(clean_images.shape).to(device)    #(64,1,28,28)

        # Sample a random timestep for each image
        t = torch.randint(0, n_steps, (n,)).to(device)

        # Computing the noisy image based on x0 and the time-step (forward process)
        noisy_images = scheduler(clean_images, t, eta)    #(64,1,28,28)

        # Getting noise prediction on the noisy images and the time-step (backward process)
        eta_theta = scheduler.backward(noisy_images, t.reshape(n, -1))    #(64,1,28,28)

        # Compare the prediction with the actual noise
        loss = F.mse_loss(eta_theta, noisy_images)

        # Store the loss for later plotting
        losses_train.append(loss.item())

        # Update the model parameters with the optimizer based on this loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Reverse process

$$x_{t-1} \approx \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

$\epsilon_{\theta}(x_t, t)$: model's output (predicted noise)

$z \sim N(0, 1)$

$$\alpha_t = 1 - \beta_t$$

$$\sigma_t^2 = \beta_t$$

```
with torch.no_grad():

    # Starting from random noise
    x = torch.randn(n_samples, c, h, w).to(device)

    for idx, t in enumerate(list(range(scheduler.n_steps))[::-1]):
        # Estimating noise to be removed
        time_tensor = (torch.ones(n_samples, 1) * t).to(device).long()
        eta_theta = scheduler.backward(x, time_tensor)

        alpha_t = scheduler.alphas[t]
        alpha_t_bar = scheduler.alpha_bars[t]

        # Partially denoising the image
        x = (1 / alpha_t.sqrt()) * (x - (1 - alpha_t) / (1 - alpha_t_bar).sqrt() * eta_theta)

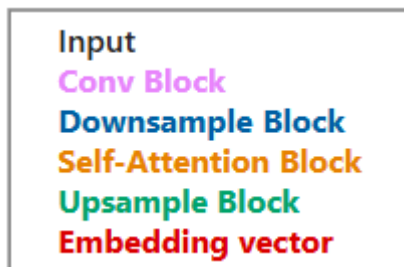
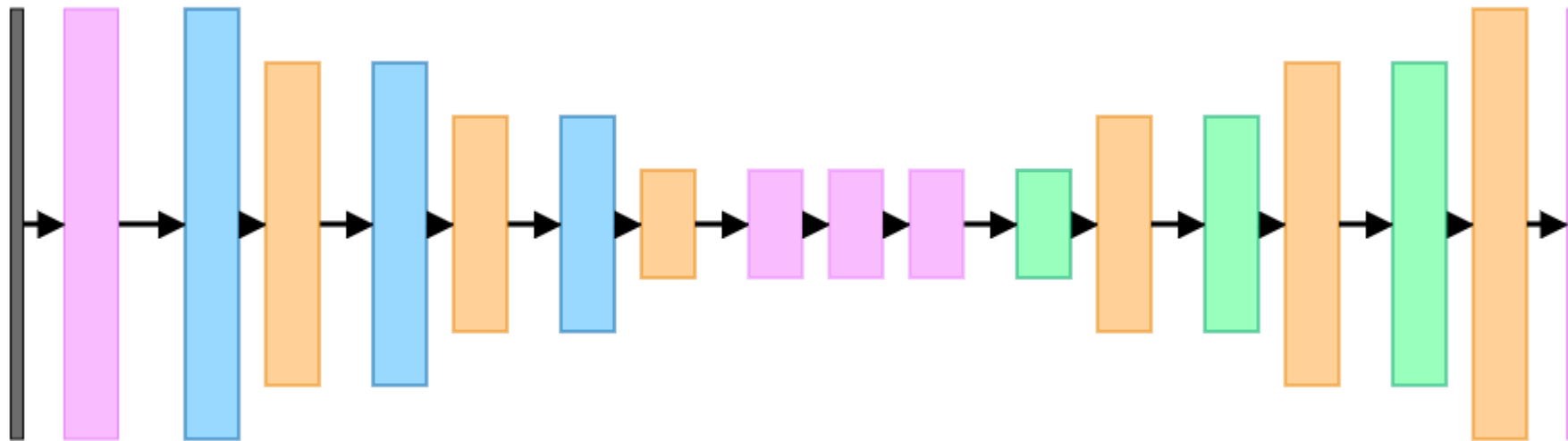
        if t > 0:
            z = torch.randn(n_samples, c, h, w).to(device)

            # Option 1: sigma_t squared = beta_t
            beta_t = scheduler.betas[t]
            sigma_t = beta_t.sqrt()

            # Option 2: sigma_t squared = beta_tilda_t
            # prev_alpha_t_bar = ddpm.alpha_bars[t-1] if t > 0 else ddpm.alphas[0]
            # beta_tilda_t = ((1 - prev_alpha_t_bar)/(1 - alpha_t_bar)) * beta_t
            # sigma_t = beta_tilda_t.sqrt()

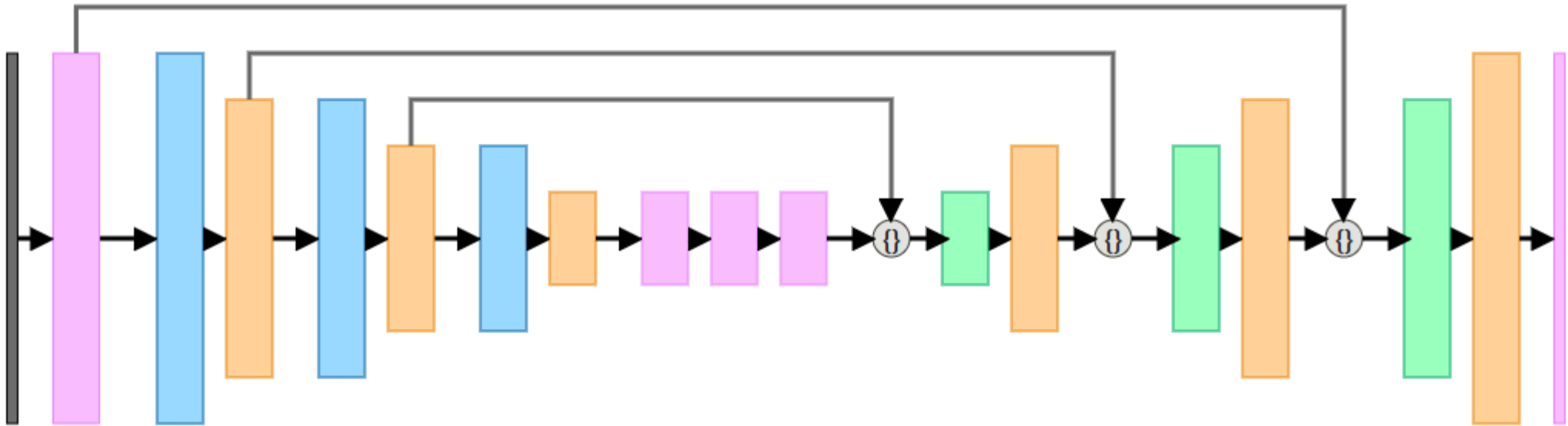
            # Adding some more noise like in Langevin Dynamics fashion
            x = x + sigma_t * z
```

Unet architecture



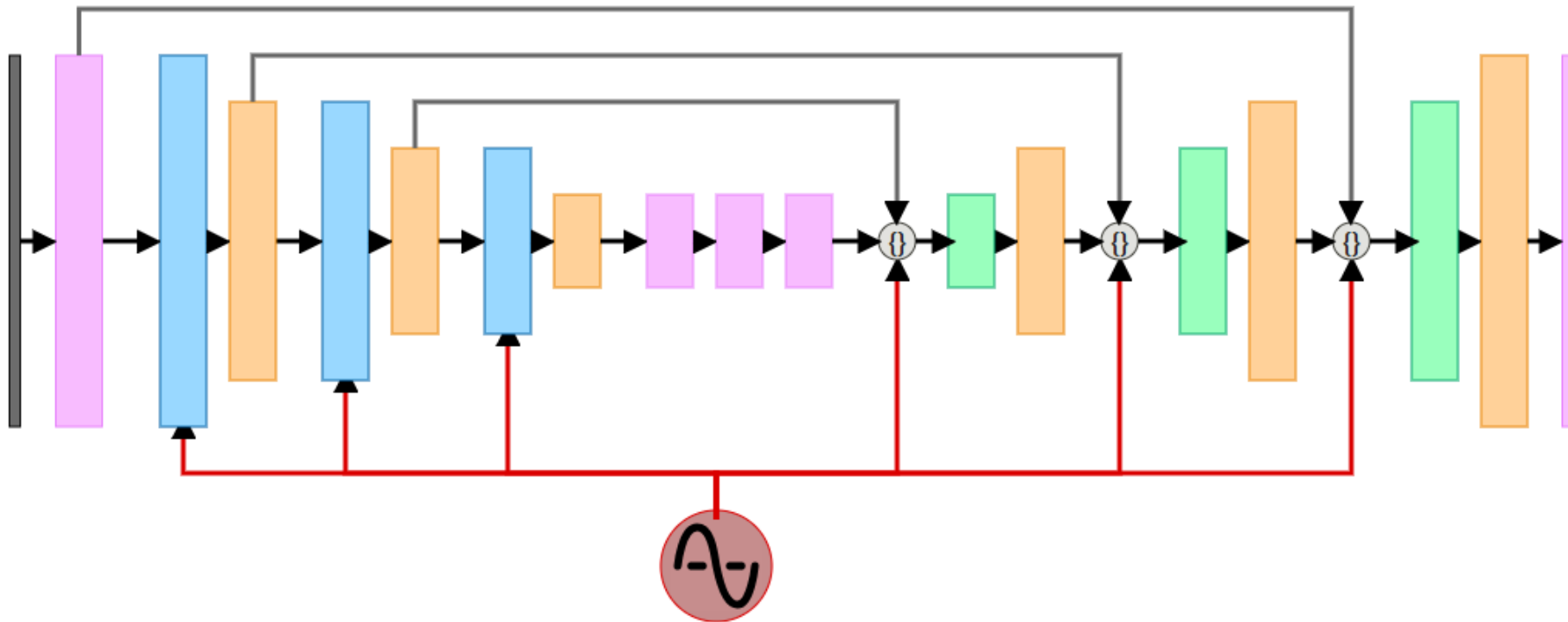
Skip connection in Unet architecture

To prevent information loss, skip connections are added to the up-sampling blocks.



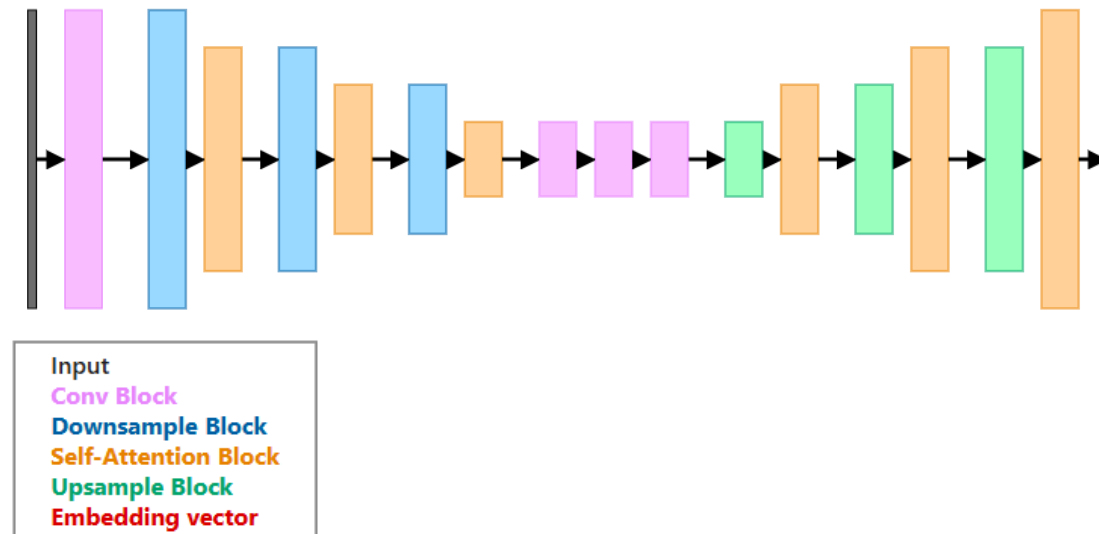
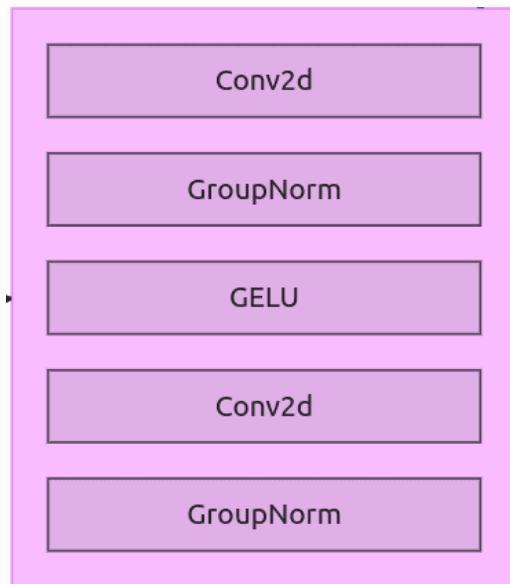
Add timestep t

to add timestep t , sinusoidal embedding is used.
It is added to all down-sample and up-sample blocks.

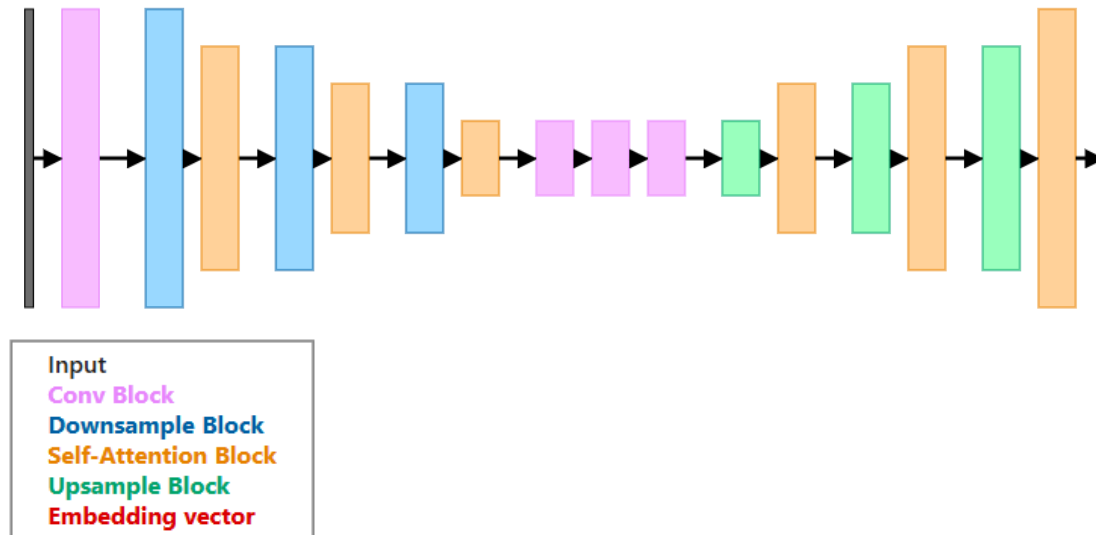
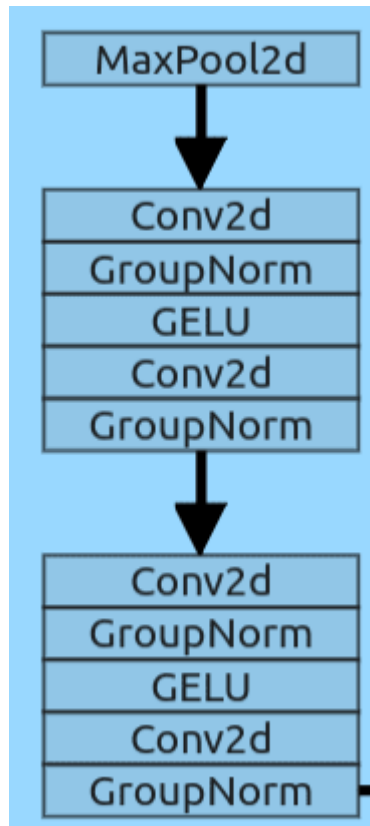


ResNet/Conv Block

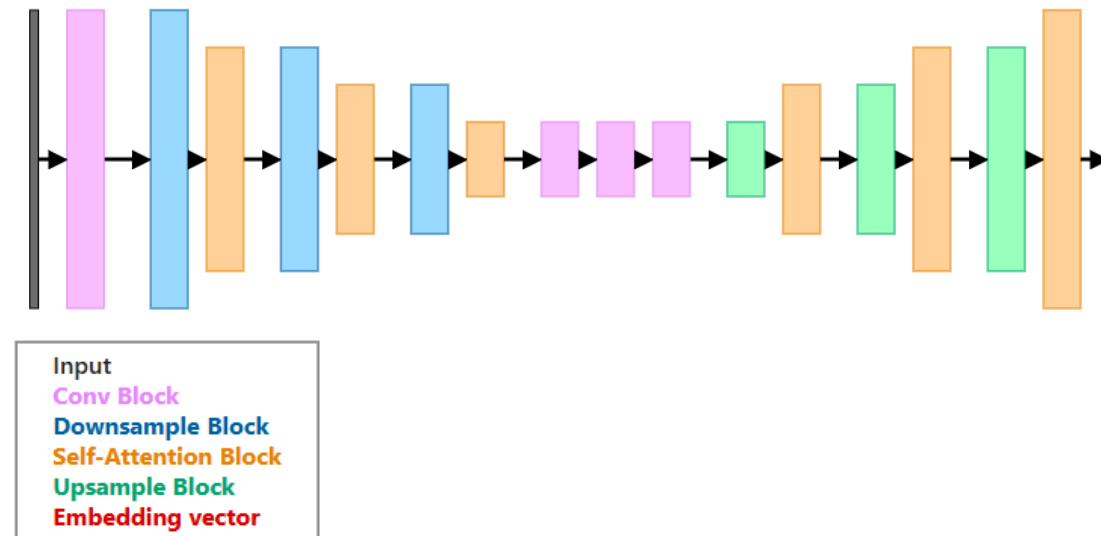
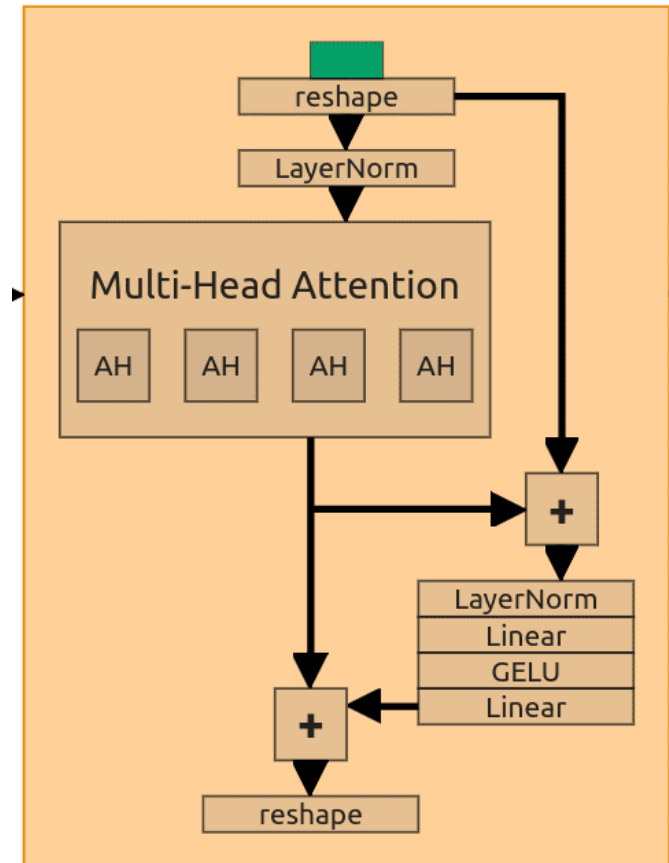
All blocks are structured in the same way, only tensor shapes are different.



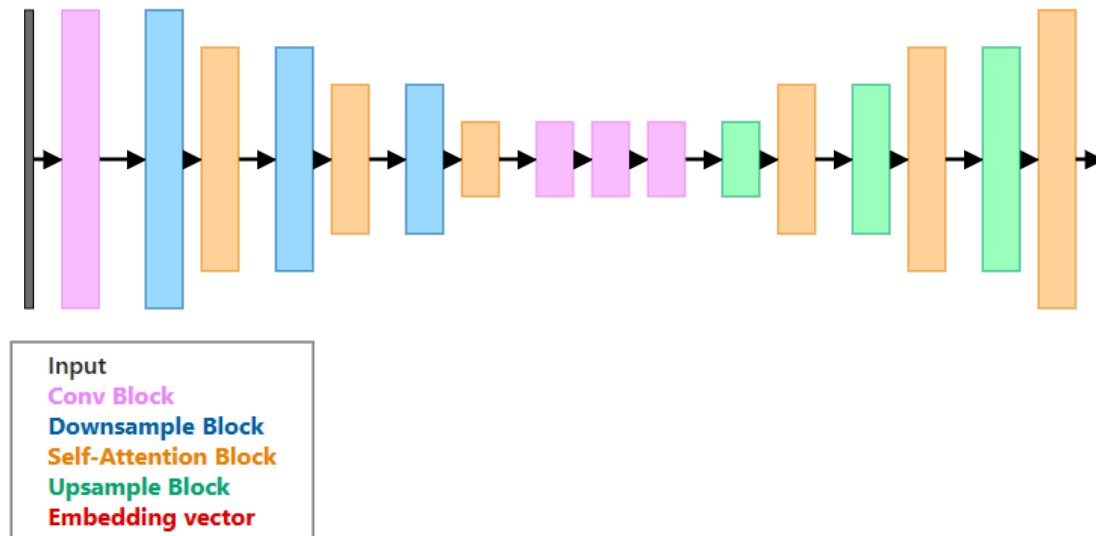
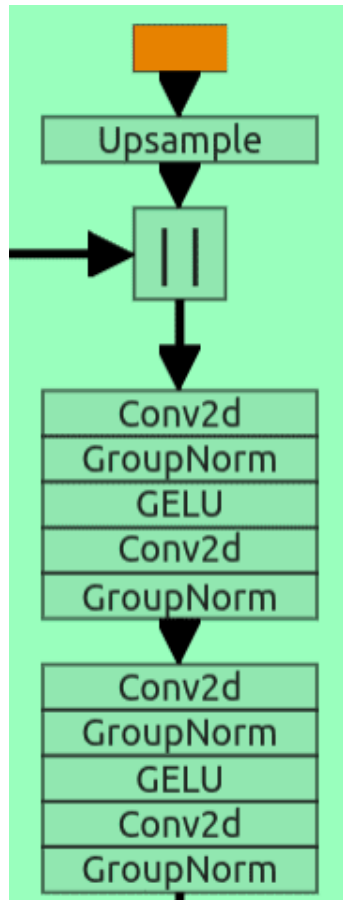
Down-sample Block



Self-Attention Block



Up-sample Block

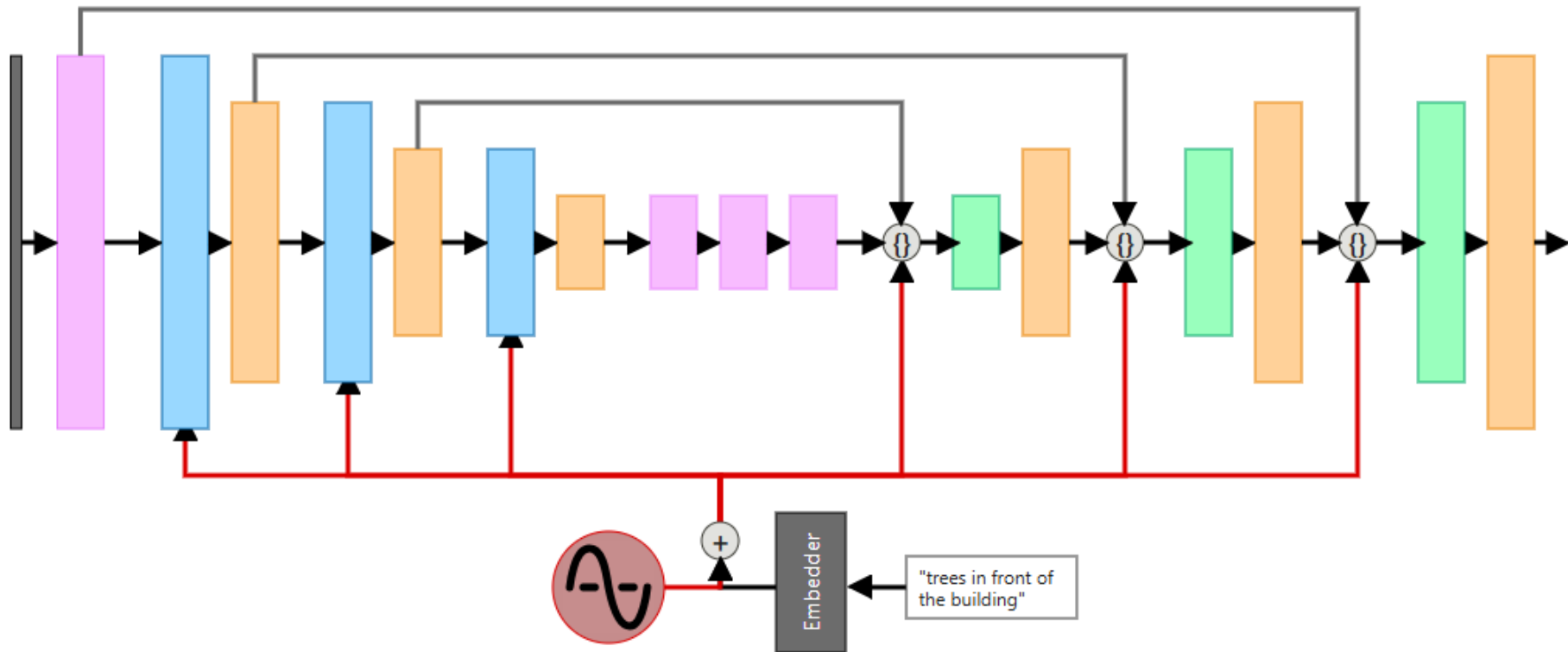


Conditional Diffusion models

Conditioning:

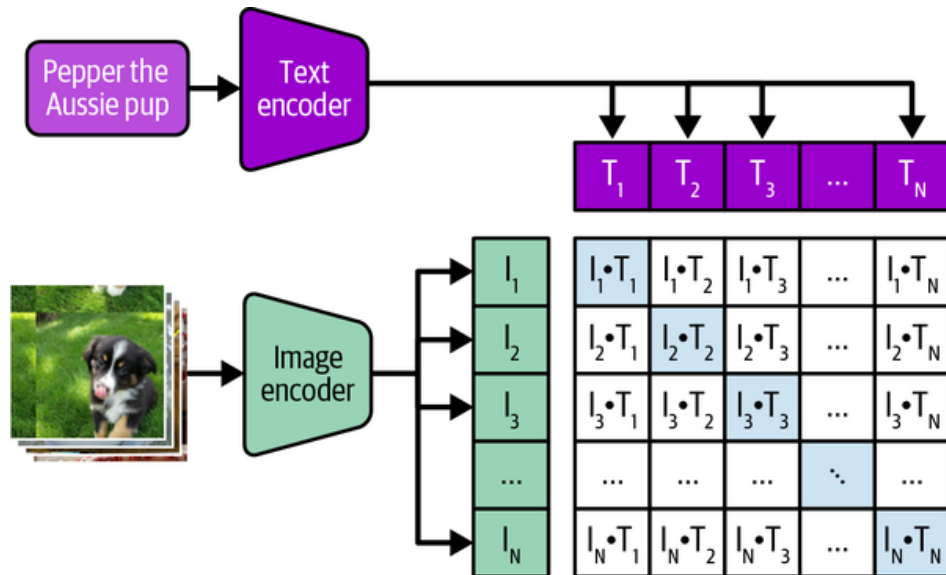
- ask the model to generate not just any image but an image belonging to a predefined class.
- guide the model's output by providing additional information, such as a label or prompt, during the generation process.

Add text embedding in the architecture



CLIP (Contrastive Language-Image Pre-training)

Measure similarity between image and text



CLIP uses contrastive loss as the loss function. It try to maximize the product of the items originally from the same pair (i.e., the blue diagonal in the image) while minimizing the rest.

CLIP

Text embedding

```
prompts = [
    "a photo of a lion",
    "a photo of a zebra",
]

# Padding makes sure all inputs have the same length
text_inputs = processor(text=prompts, return_tensors="pt", padding=True)
text_inputs

{'input_ids': tensor([[49406,  320, 1125,  539,  320, 5567, 49407],
                      [49406,  320, 1125,  539,  320, 22548, 49407]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1],
                                                                 [1, 1, 1, 1, 1, 1, 1]])}

text_inputs = {k: v.to(device) for k, v in text_inputs.items()}

with torch.inference_mode():
    text_output = clip.text_model(**text_inputs)

text_embeddings = text_output.pooler_output
text_embeddings.shape

torch.Size([2, 768])
```

CLIP

Image embedding

```
with torch.inference_mode():
    output = clip.vision_model(pixel_values.to(device))
    image_embeddings = output.pooler_output
    image_embeddings.shape

torch.Size([1, 1024])
```



Linear projection

```
print(clip.text_projection)
print(clip.visual_projection)
```

```
Linear(in_features=768, out_features=768, bias=False)
Linear(in_features=1024, out_features=768, bias=False)
```

```
with torch.inference_mode():
    text_embeddings = clip.text_projection(text_embeddings)
    image_embeddings = clip.visual_projection(image_embeddings)
    text_embeddings.shape, image_embeddings.shape
```

```
(torch.Size([2, 768]), torch.Size([1, 768]))
```


CLIP

Measure similarity between image and text

```
text_embeddings = text_embeddings / text_embeddings.norm(  
    p=2, dim=-1, keepdim=True  
)  
image_embeddings = image_embeddings / image_embeddings.norm(  
    p=2, dim=-1, keepdim=True  
)
```

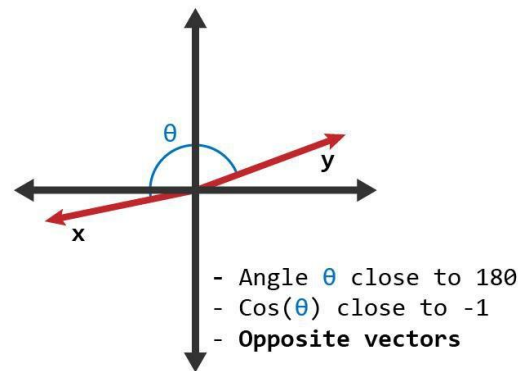
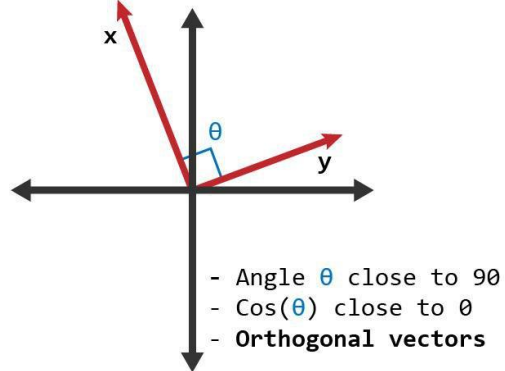
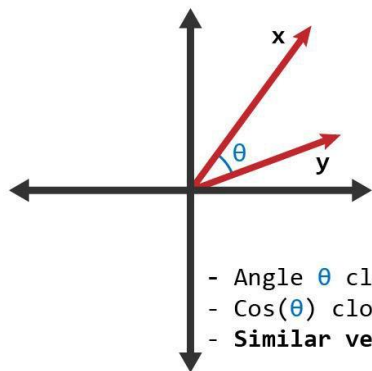
```
similarities = 100 * torch.matmul(text_embeddings, image_embeddings.T)  
similarities.softmax(dim=0).cpu()
```

```
tensor([[0.9441],  
        [0.0559]], device='cuda:0')
```

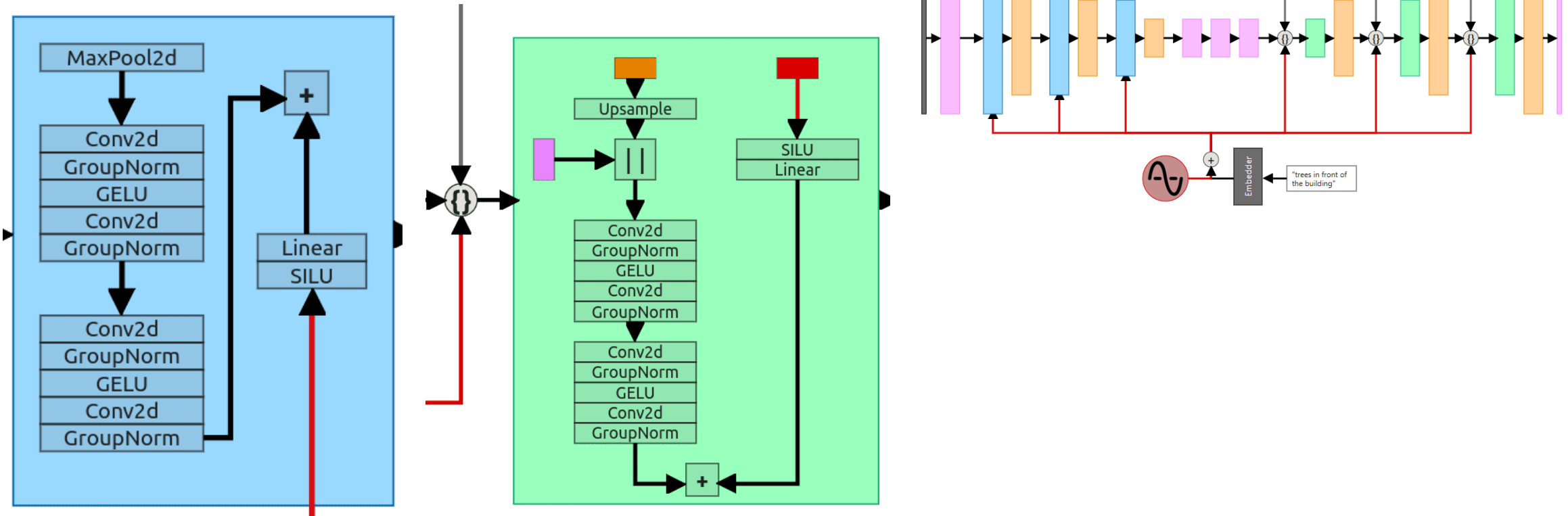
cosine similarity

$$A \cdot B = |A||B| \cos(\theta)$$

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}$$



Add text embedding in the architecture



Latent Diffusion (Stable Diffusion)

- As image size grows, so does the computational power required to work with those images.
- Latent Diffusion tries to mitigate this issue by using Variational Auto-Encoder.
- The VAE used in Stable Diffusion compress a $3 \times 512 \times 512$ input image to a $4 \times 64 \times 64$ latent.
- Using smaller images, we can get benefits for lower memory usage, fewer layers needed in the UNet, faster generation times.

